代码驱动的新手教材 / 2026-11-11 • Asia/Shanghai

一帧海洋，十二个为什么。

Ultra-Water · 从频谱到可见水面，再到浮力与证据。

中文单文件 • 12 个内联 SVG • 约 17 页 A4

范围：固定源码的静态解释，加一次已执行的安装、构建与 CPU 数值实验。协调者另完成水面/水下渲染、焦散单参数对照与一次内置基准；证据范围见实验页。GPU 指图形处理器，CPU 指通用处理器；着色器指在 GPU 上运行的计算/着色程序。

静态源码
12 问机制与边界。

本机实测
14 项自建检查通过；非官方测试。

浏览器视觉
水面/水下出图与焦散开关已由协调者实际观察。

GPU 性能
一次内置基准：2.38 ms/frame；仅该场景单次UI报告。

默认分支：master
固定 SHA： `30ea65ecb0a66b137aa200d099965634bdd84d51`
最终数值时间：2026-11-10T15:28:31.504Z（UTC）
`SUMMARY checks=14 status=PASS gpu=NOT_TESTED`

能力用途 / 5 条

1. **搭建实时海岛/海面视觉原型**：组合海浪、岸浪与天空反射。
2. **学习 GPU 频域海浪**：跟踪四级级联、IFFT 与纹理组织。
3. **研究水体光学近似**：拆分泡沫、折射、焦散和水下通道。
4. **做漂浮交互演示**：理解采样单元浮力、阻力及异步水高。
5. **练习有证据的性能分析**：区分屏幕调度、分通道计时和批量基准。

用途对应证据
README.md · L51–L59　README.md · L61–L97　README.md · L99–L117　src/main.js · L443–L480

阅读路线：管线 → 波浪模型 → 光学 → 物理与同步 → 计时 → 边界。每张问答卡可独立打印；浏览器“打印 / 另存为 PDF”即可。

01 / 一帧海水，是按什么顺序做出来的?

> **先定义 /** 渲染通道（pass）：把一项 GPU 工作画进指定目标；HDR：保留高亮范围的颜色缓冲。

这不是一个“加上水材质就结束”的示例。`renderFrame(dt)` 先取最近完成的水面查询，再推进漂浮物；接着更新漂浮物阴影、入水波纹、FFT 海浪与焦散，最后提交下一批水面查询。旧查询供当前物理使用，新查询留给之后的帧。

之后按顺序画平面反射（相机低于查询水面约 0.3 米时跳过）、不透明场景、水面；相机在水面下，或高出水面不足约 1.2 米时，再做水下合成，深潜时可以跳过近裁面水线判定，最后后处理。水面需要前面的场景颜色与深度才能表现折射；水下合成需要水面的正反面标签才能判断哪些像素在水下。先后次序本身就是数据依赖。

阅读时沿“谁写纹理 → 谁读纹理”追踪，而不是只沿场景树找对象。这里把你已有的 Three.js 场景图、先测后优化与证据优先作为基础，不再展开。



图 1 · 机制示意，不是运行截图；形状与比例不代表测量。

> 读代码时问：如果交换两个通道，哪个输入会变成旧数据或根本不存在?

固定提交证据
src/main.js · L371–L435

02 / 为什么要四级级联，而不是一张大波浪图?

> **先定义 /** FFT（快速傅里叶变换）：高效转换频域与空间域；本项目用 IFFT（逆变换）生成海面。级联（cascade）：按尺度分工的波段；图集（atlas）：把多层数据排进一张纹理。

源码固定四个周期长度：600、85、12、2.1 米，每层都是 256×256，纵向堆成 256×1024 图集。它们不是四个 FFT 计算阶段，也不是四层网格 LOD（细节层级）；每个级联负责一段波数范围，边界由下一层基频的六倍决定。

频域演化后，横向八级、纵向八级 Stockham IFFT（分阶段重排数据的逆变换）在 ping/pong 两个目标间交替读写，称为乒乓缓冲；MRT（多渲染目标，一次绘制写多份输出）把位移及导数打包计算。这里“级联数”和“变换阶段数”务必分开。

前三层贡献几何位移，第四层只贡献坡度，让细小纹理影响反光而不强迫网格追逐微波。mipmap（逐级缩小的纹理）按网格间距过滤位移，坡度二阶矩则用于粗糙度。四层索引在多处写死，并没有现成的“级联数”开关。

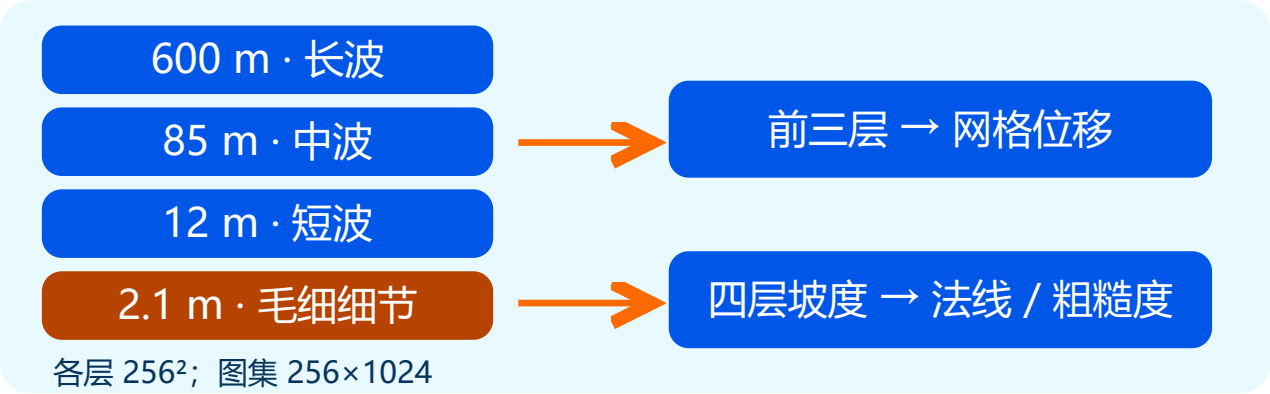


图 2 · 机制示意，不是运行截图；形状与比例不代表测量。

> 读代码时问：想移除第四层，是否同时处理法线、粗糙度及硬编码采样?

固定提交证据

src/water/OceanSim.js · L4–L19
src/water/OceanSim.js · L262–L280
src/water/OceanSim.js · L304–L347
src/water/OceanSurface.js · L299–L344
src/water/OceanSurface.js · L428–L458

03 / JONSWAP 到底决定了海浪的什么?

> **先定义 /** 频谱：能量如何分布到不同频率；JONSWAP：描述有主峰的风浪能量谱模型。风区长度（fetch）：风沿海面持续作用的距离；涌浪（swell）：与本地风浪分开描述的长周期波。

频谱决定“哪些频率强”，不是直接画出某个浪头。`jonswapParams` 用风速与风区长度求 alpha 和峰值角频率；`jonswap` 用峰值两侧不同的宽度，以及 gamma 的峰增强项，塑造能量曲线。提高 gamma 更像突出主峰，不等于整体抬高所有波浪。

初始化循环叠加两套谱：风浪与涌浪。方向分布通过余弦的 2s 次幂等表达式决定能量朝哪里散开；短波衰减抑制过细波段。最后乘高斯随机数，产生固定种子下的初始复振幅。

从角频率谱转到二维波数网格时，代码还乘 dω/dk、除以 k，再乘网格间隔平方；不是把一条谱曲线直接当高度贴图。要改变海况，先辨认在改总强度、主峰、方向还是细节衰减。

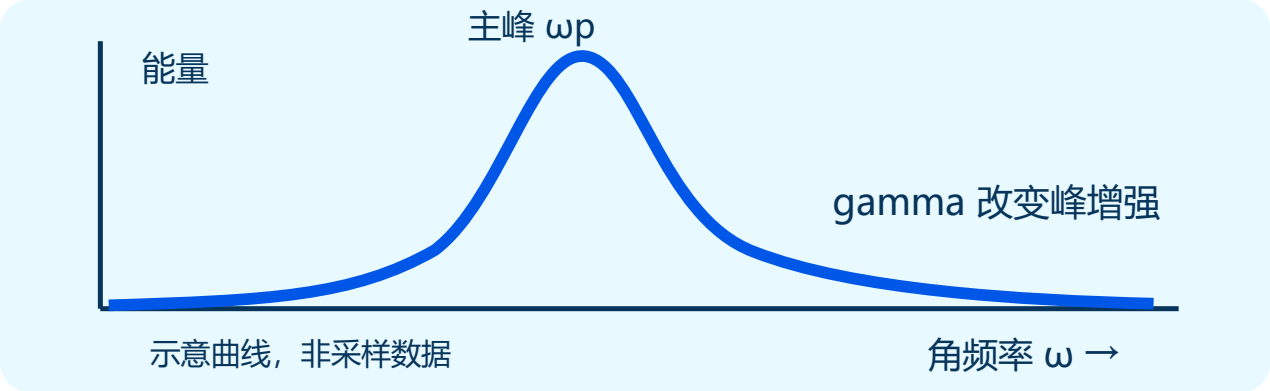


图 3 · 机制示意，不是运行截图；形状与比例不代表测量。

> 读代码时问：风向、涌浪方向和 gamma 各自改动了哪一项?

固定提交证据

src/water/OceanSim.js · L50–L100
src/water/OceanSim.js · L197–L226
src/water/OceanSim.js · L289–L301

04 / 不同波长为什么走得不同？循环又意味着什么？

> **先定义 /** 波数 k = 2π/波长；角频率 ω：相位每秒变化的弧度。色散关系：ω 与 k 的对应关系；相位：波在周期中的位置。

公共 GLSL 中统一使用 `ω² = gk + (σ/ρ)k³`。第一项是重力贡献，第二项是表面张力贡献；细波的后者更重要。不是只有第四层调用这条公式，只是第四层专门承担重力—毛细细节。相速度为 ω/k，因此不同尺度不会像一张整体平移的贴图那样同速。

演化时并不直接使用连续 ω：代码把它向下量化到 `2π/300` 的整数倍；时间按 300 秒取模，再把相位约化进一个周期。这样 FFT 的相位重复，并避免长时间累积导致单精度相位精度变差，但频率也有量化误差。

这个循环承诺只针对 FFT 相位，不是整段场景视频。泡沫有历史恢复，物理有状态；岸浪、噪声和表面时间也另走时钟。不能从“FFT 循环”推断漂浮物、泡沫及所有后处理都能无缝循环。

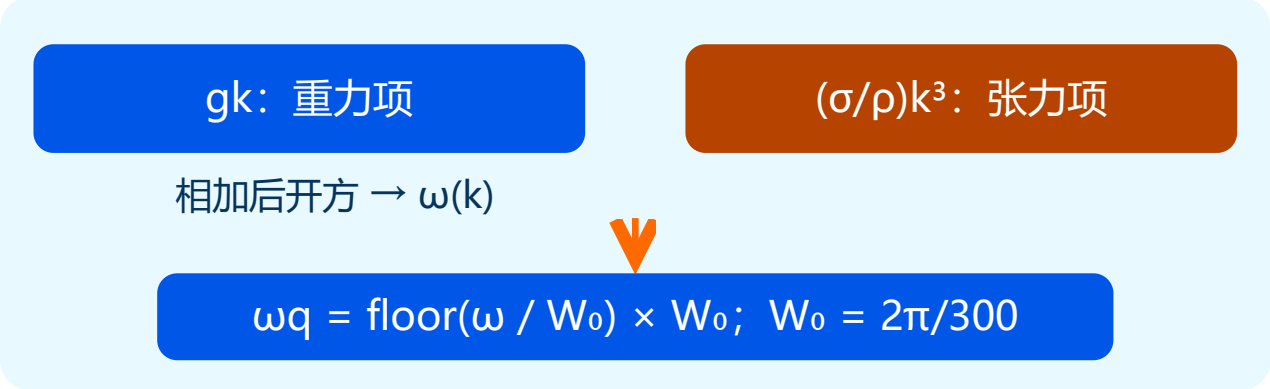


图 4 · 机制示意，不是运行截图；形状与比例不代表测量。

> 读代码时问：验证的是 FFT 纹理相位，还是包含所有状态的最终画面？

固定提交证据

src/water/OceanSim.js · L14–L30
src/water/OceanSim.js · L122–L141
src/water/OceanSim.js · L304–L310
src/main.js · L382–L396

05 / 浪峰为什么变白，Jacobian 又是什么?

> **先定义 /** Jacobian（雅可比行列式）：局部坐标变形的面积比例；此处描述水平位移把海面挤紧还是拉开。λ 是水平尖锐位移的强度，不是波长。

组装通道算 `J=(1+λDxx)(1+λDzz)-(λDxz)²`。J 下降表示水平压缩增强；相比只用“高度很高就涂白”，它能更贴近尖锐浪峰处的破碎倾向。但它是视觉判据，不是气泡流体求解。

位移纹理的 alpha 存的是带恢复历史的 turbulence：`min(J, prev + dt×recovery/max(J,0.5))`。压缩发生时立即下降，之后按时间步恢复。表面再把前三层该值按水深加权，经过 bias/scale 阈值转换为泡沫量；不是直接把精确总位移的 Jacobian 重算一次。

最终白沫还叠加岸浪破碎、浅水接触等来源，使用花边/团块纹理、覆盖侵蚀和凹凸着色。恢复速度、几何尖锐度、着色阈值是不同控制项，别把纹理变白误解为真实空气被模拟出来了。

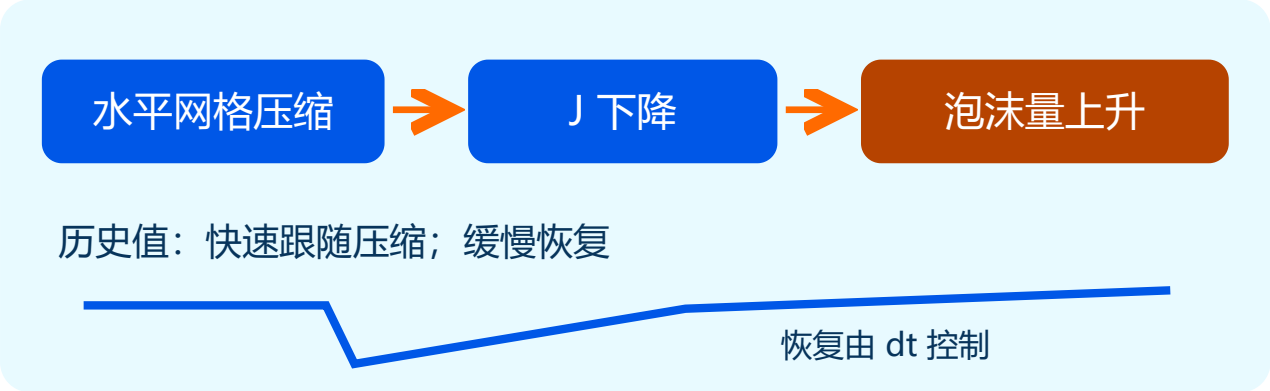


图 5 · 机制示意，不是运行截图；形状与比例不代表测量。

> 读代码时问：白色来自 FFT 压缩，还是岸浪/接触泡沫?

固定提交证据

src/water/OceanSim.js · L169–L195
src/water/OceanSurface.js · L464–L467
src/water/OceanSurface.js · L647–L710

06 / 为何正看能见底，掠看却更像镜子?

> **先定义 /** 菲涅耳（Fresnel）：界面反射比例随观察角变化；折射：光穿过介质界面时转向。Beer–Lambert 吸收：光随介质路程按指数衰减。

水面使用粗糙度（微小法线分散程度）感知的菲涅耳近似：`F=F0+(F90−F0)(1−N·V)⁵`，F0 固定为 0.02，F90 随粗糙度调整。掠看时反射增强，但这里并非粗糙远海也强行变成纯白镜面。N 是单位法线，V 是朝向相机的单位向量。

水上折射是屏幕空间近似：由法线偏移场景颜色采样坐标，用深度差估计穿水厚度。如果偏移采到了水面前的物体，就平滑回退，减轻物体边缘错误；颜色再按 `T=exp(−吸收系数×厚度)` 衰减并加入散射。它不是完整三维光线追踪。

反射组合天空立方体（六面环境纹理）和以海平面 y=0 镜像相机得到的平面反射，按粗糙度采样模糊层级。最后混合水体颜色和反射，再加太阳高光。图里的光线说明角度关系，实际水上折射实现仍是屏幕偏移。

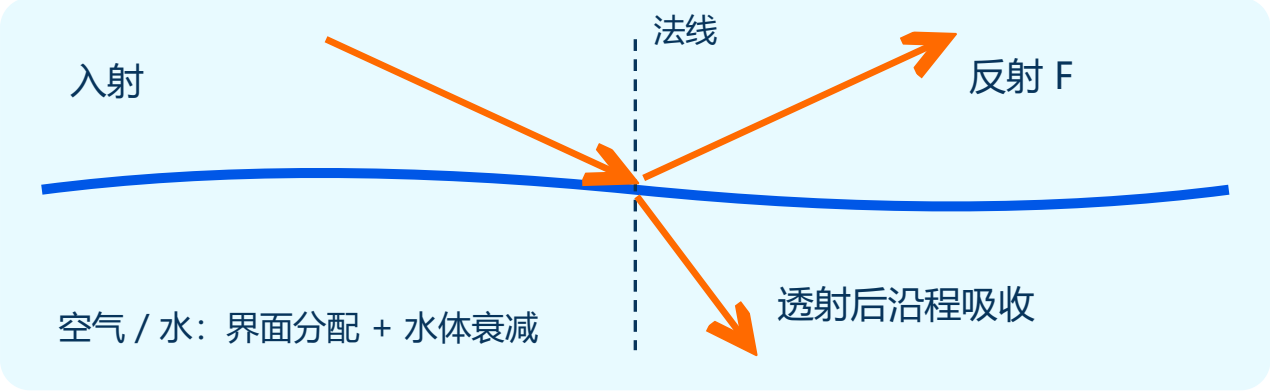


图 6 · 机制示意，不是运行截图；形状与比例不代表测量。

> 读代码时问：错误出在角度权重、深度厚度，还是屏幕外信息不可用?

固定提交证据

src/water/OceanSurface.js · L508–L521
src/water/OceanSurface.js · L561–L602
src/water/OceanSurface.js · L639–L645
src/engine/PlanarReflection.js · L15–L54

07 / 海底的亮线是怎样聚出来的?

> **先定义 /** 焦散（caustics）：折射光束汇聚形成的明暗纹；面积比铺点（splatting）：把源面积映到接收面，用面积缩放分配亮度。

焦散只读取 12 米级联的坡度，不是四层完整海面光线追踪。顶点把太阳方向按水的折射率折入水下，投到 1.5、4、10 米三个深度切片；减去平水面落点偏移，让纹理坐标对准入水点。

同样多的入射光落到更小区域就更亮。片元通过屏幕导数求原面积/落地面积，并用加法混合叠加重叠三角形。坡度用三次 B 样条平滑采样，目标使用 MSAA（多重采样抗锯齿），避免细光线断成点。

查询时按深度插值切片，深处增加模糊，最终作为直射太阳光倍率。理想构造意图是重分配能量、均值约一；但源码把面积倍率截到 16，落点也有边界夹取，故不能宣称实现了严格能量守恒。低太阳角尤其应检查这些近似。

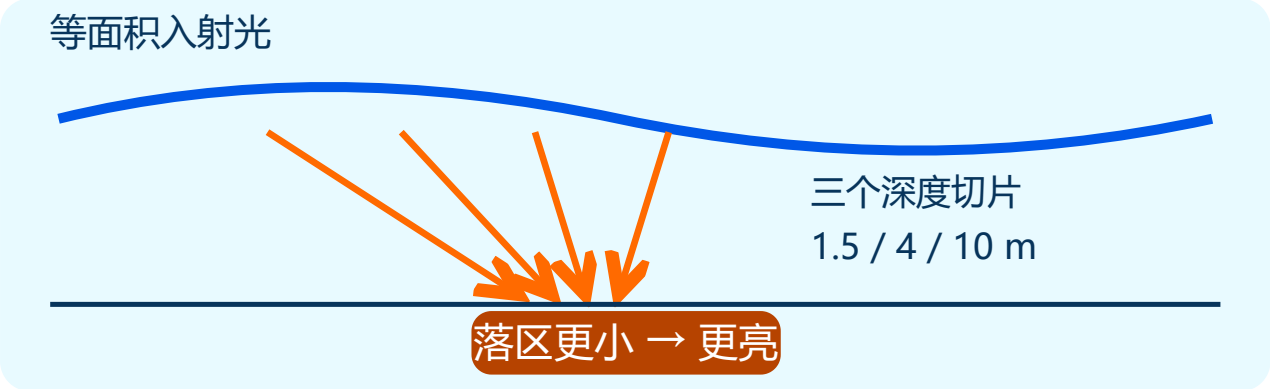


图 7 · 机制示意，不是运行截图；形状与比例不代表测量。

> 读代码时问：变亮只是空间汇聚，还是截断/采样造成的异常热点?

固定提交证据

src/water/Caustics.js · L5–L35
src/water/Caustics.js · L47–L106
src/water/Caustics.js · L116–L134
src/water/Caustics.js · L143–L159

08 / 相机半入水时，为何不能只加一层蓝雾?

先定义 / Snell 窗：水下向上看时能透见水上世界的角度窗口；全内反射：超出临界角后光不能穿出水面。弯月线（meniscus）：此处指镜头水线附近的暗边效果。

水面是双面着色。背面分支折射天空、读取水上物体，并在 Snell 窗外近似反射深水颜色；临界角边缘按粗糙度柔化。它不是把整幅图统一调成青色。

水下合成先逐像素判断：水面 alpha 标签区分顶面与底面；看不到水面的地方，用 64×36 镜头近裁面水深图判定。这样浪冲过镜头时，同一帧中可以上半空气、下半水体，并在正反面交接处加暗水线。

水下路程产生指数消光与入散射。太阳光柱在半分辨率下沿视线做 24 个带抖动的单次散射采样，同时读取焦散与地形/漂浮物阴影，再按深度一致性放大合成。单次散射指只计算一次散射事件，不是完整多次散射体积输运。



图 8 · 机制示意，不是运行截图；形状与比例不代表测量。

读代码时问：水线归属错误，还是水下消光/光柱错误?

固定提交证据

src/water/OceanSurface.js · L469–L505
src/water/UnderwaterPass.js · L27–L94
src/water/UnderwaterPass.js · L112–L184
src/water/UnderwaterPass.js · L187–L225
src/main.js · L398–L429

09 / 箱子为什么会摇，而不只是跟着高度上下动?

> **先定义 /** 浮力：排开水的重量产生的向上力；刚体：形状不变、可平移和旋转的物体；力矩 τ = r×F：偏离质心的力引起转动。

每个漂浮物被描述成多个采样单元。单元位置与水高比较，得到浸没比例；浮力为 `ρg×单元体积×浸没比例×倍率`。在各单元施力，左右或前后受力不均就自动出现横摇和俯仰，而不是额外套一段摇摆动画。

阻力使用单元速度减局部水速度；水速度从多次查询得到的位移差估计，并被限幅，以免陡峭波前扫过时夸大粒子速度。积分以不大于约 1/240 秒的子步推进，单次输入 dt（帧时间间隔）还被限制。子步更细不意味着水面查询也变成同频新数据。

触底用地形高度弹簧及摩擦近似，转动惯量用实心盒近似。漂浮物入水可产生涟漪和飞溅，但没有物体间碰撞，也没有持续航行尾流；它是演示级波浪—刚体耦合，不是完整航海工程模拟器。

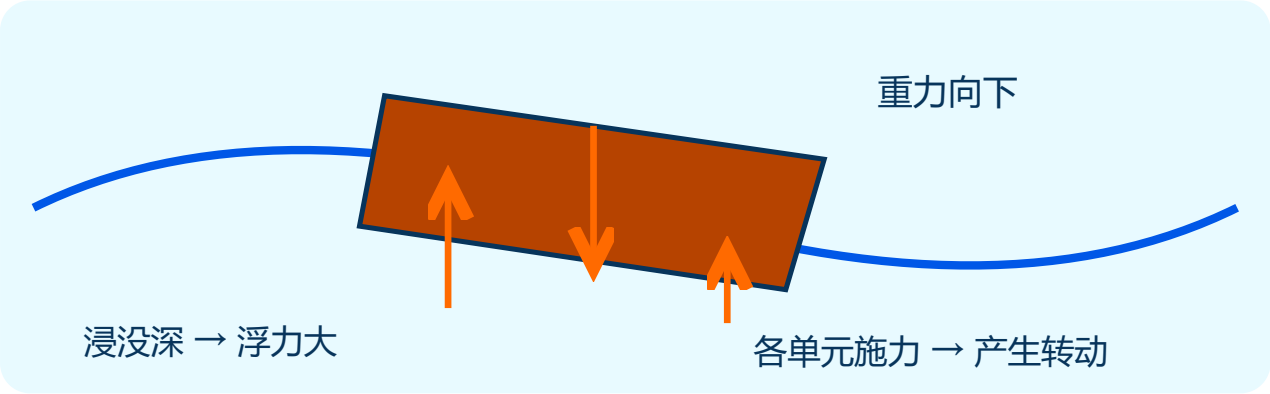


图 9 · 机制示意，不是运行截图；形状与比例不代表测量。

> 读代码时问：稳定性取决于子步、阻力，还是延迟的水面数据?

固定提交证据

src/scene/floaters.js · L4–L38
src/scene/floaters.js · L111–L178
src/scene/floaters.js · L181–L201
README.md · L151–L156

10 / GPU 上的海浪，CPU 怎样读而不卡住?

> **先定义 /** 异步回读：提交 GPU 数据读取后不立刻等待，完成后再消费；PBO：像素缓冲对象；fence：标记 GPU 命令完成位置的同步栅栏。

`WaterQuery` 把待查询世界 xz 坐标装进纹理，画一个窄条目标，再调用 Three.js 的 `readRenderTargetPixelsAsync`。本应用没有手写 PBO/fence 回读实现；PBO 与 fence 是 README/注释对底层机制的说明，依赖实现应在依赖源码中另核。

查询着色器复用 `waterAt`：对水平尖锐位移做四次固定点反解，再采样高度。main 把容量设为 512（类默认是 256），最多允许两批在途；容量满会截断，已有两批时本次直接跳过，不是每帧保证交付。结果包含高度、水平位移、时间及布局标签。

物理只接收匹配布局的结果，容忍注释所说的若干帧延迟。注意“复用同一函数”不等于逐三角形绝对相同：查询用最高细节，而可见网格会过滤位移，并在顶点阶段作海床下限修正；近岸必须单独核验查询与视觉偏差。异步减少同步等待，不保证零成本或零延迟。



图 10 · 机制示意，不是运行截图；形状与比例不代表测量。

> 读代码时问：回来的点属于哪个布局、哪个时间？是否超出容量?

固定提交证据

src/water/WaterQuery.js · L5–L58
src/main.js · L83–L88
src/scene/floaters.js · L111–L135
src/water/OceanSurface.js · L299–L355

11 / HUD、GPU 计时和不封顶循环，测的是同一件事吗?

> **先定义 /** vsync（垂直同步）：按显示刷新节奏调度；timer query：由 GPU 记录一段命令的耗时；disjoint：GPU 计时不连续、结果可能无效的状态。

普通循环用 requestAnimationFrame 调度，`tick` 限制 dt、更新控制器和相机，再画一帧。HUD 用时间窗口内的帧计数估算频率，它描述该调度模式下的吞吐，不等于显示器实际呈现了同样多张独立画面。

`GpuTimer` 使用可选的 EXT_disjoint_timer_query_webgl2 扩展，不可嵌套；新标签会先结束旧标签。它延后查询可用性，常规路径遇到 disjoint 不更新统计，结果按标签平滑。扩展缺失时这些分项不可用，不能拿 CPU 提交时间冒充 GPU 时间。

不封顶模式用 MessageChannel 连续调度，以 fence 将在途帧限制在三帧；隐藏标签页回到 rAF。按需基准则暂停普通循环，预热后批量渲染，等待 fence 确认完成再除以帧数，并读出分项。`drain()` 本身未重做 disjoint 检查，采证时仍应记录计时有效性。本次只执行一次该基准，原始UI报告及局限见实验页；未独立核查该批计时的disjoint状态，不能据此作性能保证。

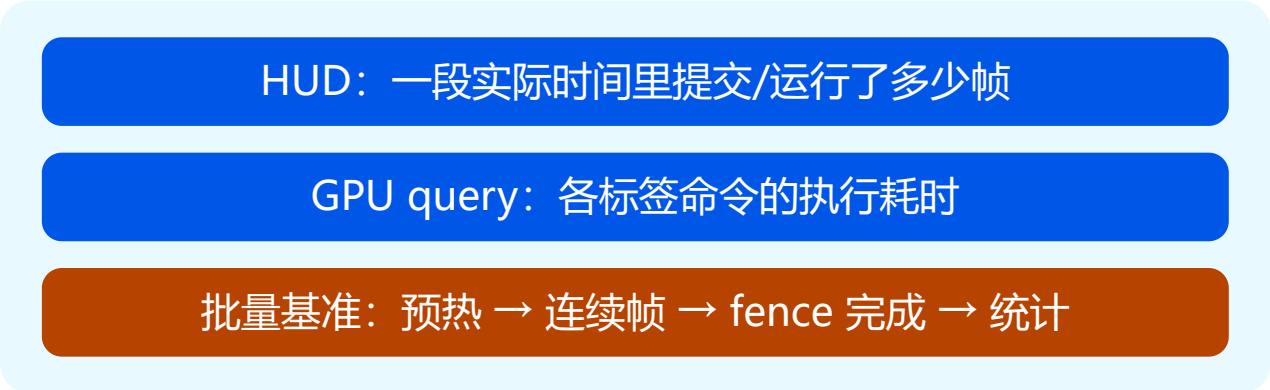


图 11 · 机制示意，不是运行截图；形状与比例不代表测量。

> 读代码时问：比较的设备、分辨率、画质、调度与热状态是否相同?

固定提交证据

src/engine/gpuTimer.js · L1–L69
src/main.js · L443–L475
src/main.js · L510–L535
src/main.js · L565–L609

12 / 这个项目的边界在哪里，什么不能据此宣称?

> **先定义 /** 高度场（heightfield）：每个水平位置只对应一个表面高度，不能表达翻卷成管的多层水面；带限：只保留采样能表达的频率，避免走样。

岸浪不是把深海 FFT 一路推到沙滩。浅水处长涌浪淡出，烘焙的传播时间场接手；其浅水速度用 √(g×深度)，深度被夹在 0.3–14 米。一阶扫掠与模糊近似传播前沿，波高及水平挤压预算限制破碎形态，所以能倾斜却不会真正卷成浪管。

空间也有界：海面是随相机移动的极坐标网格，远处接平裙边；岸线图超出范围时按深水处理。FFT 纹理按各自周期重复，不等于物理无限且永不重复的大海；焦散只有三个深度切片，水上折射只知道当前屏幕数据。

明确未实现：持续航行尾流、漂浮物互撞、真实翻卷浪管（README 已列）。本次所读核心链路未找到完整体积流体求解，不能把它当工程预测器。WebGL2、浮点渲染目标及具体驱动支持要在目标机确认。本机安装、生产构建及 CPU 数值检查已完成，协调者观察到服务器启动和 Lagoon 渲染；这些都不能替代完整的 GPU 兼容性、长期稳定性或性能测试，本文不宣称稳定帧率或长期无漂移。



图 12 · 机制示意，不是运行截图；形状与比例不代表测量。

> 读代码时问：是模型未覆盖，还是数值/采样近似造成的边界?

固定提交证据

src/water/shoreMap.js · L75–L86
src/water/shoreMap.js · L258–L271
src/water/OceanSurface.js · L5–L42
src/water/OceanSurface.js · L55–L58
src/water/OceanSurface.js · L201–L275
README.md · L151–L156

先锁版本，再锁依赖并构建

命令口径：第1–4步保留原Git Bash实跑命令；第5步保留协调者已执行的PowerShell启动命令。首次取得固定源码与解压位置见exercise/README.md。

1 / 锁定 SHA 与环境

```
cd /f/reminder/ultra_water
date -Iseconds
node --version
npm --version
git -C repo rev-parse HEAD
git -C repo status --short
```

目的：把源码、锁文件与输出绑定到同一提交，并检查 Node 是否满足 Vite 8.3.5 的版本范围。

真实结果 • 2026-11-10T23:19:54+08:00：Node `v24.21.0`，npm `11.19.0`；HEAD 为 `30ea65ecb0a66b137aa200d099965634bdd84d51`。最终 `status --short` 无输出，表示无 tracked 上游改动。

验收：SHA 必须逐字符一致；Node 满足 `^20.19.0 || >=22.12.0`；状态输出为空。退出码均为 0。

2 / 锁定依赖、安装并生产构建

```
cd /f/reminder/ultra_water/repo
npm ci --ignore-scripts --no-audit --no-fund
npm ls --depth=0
npm run build
find dist -maxdepth 2 -type f -printf '%P %s bytes\n' | sort
```

目的：只用上游 lock 安装最小依赖，关闭生命周期脚本，再执行真实 Vite 构建并核对许可产物。

真实结果：安装 2026-11-10T23:21:07–23:21:16+08:00，`added 17 packages in 8s`；顶层为 `three@0.186.1`、`lil-gui@0.21.0`、`vite@8.3.5`。构建于 23:21:24–23:21:26，32 个模块、227 ms、退出码 0；主 JS 751.48 kB，仅有超过 500 kB 的非阻断提示。`licenses.md` 2293 bytes，`THIRD_PARTY_NOTICES.md` 2700 bytes。

验收：安装、依赖树、构建均退出 0，许可文件尺寸如上。构建只证明打包，不证明 shader、海面或 GPU 性能通过；`--no-audit` 也不表示依赖无漏洞。

原始证据（只读）

F:/reminder/ultra_water/exercise/experiment-log.txt · [2]–[4]

执行上游参数，再做 CPU 数值对照

3 / 上游 JS 参数与边界验证

```
cd /f/reminder/ultra_water
node --check exercise/verify-ocean.mjs
node exercise/verify-ocean.mjs
```

目的：直接导入上游 `N`、`CASCADE_LENGTHS`、`defaultOceanParams`；在 VM 中只移除 import/export 语法，执行私有 `jonswapParams`，观察正常参数和非法输入。

真实结果： `jonswapParams(9,120)` 返回 `alpha=0.00922769519522109`、`peakOmega=1.013664848341946`。更强风使 alpha 上升、peakOmega 下降。零/负风速或 fetch 至少一个参数非有限；JSON 中的 `null` 是 NaN/Infinity 序列化结果，不是有效的 0。四级 texel 尺寸依次为 2.34375 / 0.33203125 / 0.046875 / 0.008203125 m。

验收：语法检查先通过；检查 01–07 全部 PASS，非法边界被如实记录而非包装成有效输入。UI 范围约束不能当作纯函数输入校验。

4 / CPU 翻译 GLSL，与独立 DFT 对照

```
cd /f/reminder/ultra_water
set -o pipefail
node exercise/verify-ocean.mjs 2>&1 | tee exercise/verification-output.txt
```

目的：按上游 GLSL 公式编写 CPU 解释器，核对色散、300 秒量化相位、Stockham 索引、centered-k 反移位、Hermitian 对称、pack2 与 Jacobian/泡沫恢复；FFT 对手是独立朴素二维逆 DFT，不是同算法副本。

真实结果 • 2026-11-10T15:28:31.504Z： `SUMMARY checks=14 status=PASS gpu=NOT_TESTED`。N=4、8 的 Stockham/DFT 最大复数误差分别为 `3.9533917949896255e-15`、`1.7245305484923l8e-14`；pack2 重建误差 `7.549516567451064e-15`；Hermitian 最大残余虚部 `8.118657102639939e-15`。固定 J 下分 6 或 60 步的泡沫恢复结果一致，并受当前 J 上限截断。

失败与修正：开发中有三次断言失败：模板文本锚点未先展开；泡沫 oracle 把应被 J=0.8 截断的结果误写为 0.86；独立 DFT 漏掉 `(-1)^(x+y)` 反移位。分别修正锚点、独立期望和参考 DFT，保留负对照，未放宽容差或改上游源码。

验收：脚本非零即失败，Git Bash启用 `set -o pipefail` 并检查退出码；14 项全过且误差达到上述量级。它们是这个练习的自建检查，**不是官方测试，也不是 GPU 证明**：不覆盖 shader 编译、浮点目标、fp16、MRT、画面或性能。

原始证据（只读）

F:/reminder/ultra_water/exercise/verification-output.txt · 第 1–20 行；experiment-log.txt · [5]–[6]

已执行实验 / 第5步 • 2026-11-11

真实出图：焦散开关与水下视角

```
Set-Location F:\reminder\ultra_water\repo
npm.cmd run dev -- --host 127.0.0.1 --port 5187 --strictPort
# 浏览器打开 http://127.0.0.1:5187/
```

目的：让固定源码在本机出图，只改一个光学强度，检查机制是否具有可见效果。协调者使用 Chrome 页面真实操作并查看截图；未在页面注入测试脚本。

1. 保持默认 **Lagoon**、1660×1053、4×MSAA、desktop profile。HUD 显示 NVIDIA GeForce RTX 4070。展开 Water，caustics=1 时，浅水沙底能看到细密网状亮纹。
2. 只将 **caustics 输入框改为0并回车。**读取输入框确认值0，下一帧截图中网状亮焦散消失，海浪、沙纹仍存在。
3. 恢复 **caustics=1**，再选择 camera=Underwater。实际画面显示蓝绿色水下雾、底部焦散和物体轮廓。
4. 在切水下之前，Lagoon 状态点击一次 **Run benchmark**，等 Benchmarking… 恢复。页面返回 **2.38 ms/frame = 420 fps，GPU 2.34 ms。**这是内置300帧暂停常规循环、连续推进300帧基准的一次观察，不是显示刷新率或稳定性能保证。

真实结果：本地水面/水下均有画面，caustics=1→0 的效果符合该光学通道的用途；恢复后水下焦散可见。新建实验页受控日志读取无 warn/error 条目；早先标签出现过着色器精度警告，不能据此声称所有运行都无警告。

CPU检查补充边界：第03项是源码字符串检查；N=4/8不覆盖256×1024图集隔离。泡沫分步样例固定J且两路最终均饱和到0.8，不能据此验证动态泡沫。

验收与范围：实际参数值与画面变化相符。时间仍在流动，不是同一帧像素差分；未验证物理精度、图集隔离或MRT/fp16误差。普通后台HUD约1FPS，受后台调度影响，不能代替基准。截图在本次工具记录中已查看，本包不声称含本地截图文件。

原始观察与数据

exercise/browser-experiment-evidence.txt；完整分项计时与限制保留。可复现步骤使用页面控件，不要求修改项目源码。

来源索引 / 固定版本

每个解释都能回到代码

官方仓库：github.com/Humpalumps/Ultra-Water
默认分支 master；本次读取其 HEAD 后锁定：
30ea65ecb0a66b137aa200d099965634bdd84d51

证据分工：固定源码解释来自教材阅读；本地安装、构建和 CPU 数值实验由 claude_kimi 执行；服务器启动、水面/水下渲染、焦散A/B及一次内置基准由协调者实际观察并记录。独立审核另见review-log，不能把本教材当作GPU正确性认证。

来源边界：README 作为功能意图；关键行为以实现为依据。PBO/fence 的底层回读实现未在应用层展开；未对依赖源码作背书。2026-11-10 的本机实测覆盖锁依赖安装、生产构建、上游 JS 参数/边界和 CPU GLSL/独立 DFT 对照；协调者于次日运行一次内置暂停常规循环、连续推进300帧基准，未做跨设备性能验证。README 的性能表未当作本地证据引用。

全部引用代码路径与行号

- `src/main.js`
  L83–L88 · L371–L435 · L382–L396 · L398–L429 · L443–L475 · L443–L480 · L510–L535 · L565–L609
- `src/water/OceanSim.js`
  L4–L19 · L14–L30 · L50–L100 · L122–L141 · L169–L195 · L197–L226 · L262–L280 · L289–L301 · L304–L310 · L304–L347
- `src/water/OceanSurface.js`
  L5–L42 · L55–L58 · L201–L275 · L299–L344 · L299–L355 · L428–L458 · L464–L467 · L469–L505 · L508–L521 · L561–L602 · L639–L645 · L647–L710
- `src/engine/PlanarReflection.js`
  L15–L54
- `src/water/Caustics.js`
  L5–L35 · L47–L106 · L116–L134 · L143–L159
- `src/water/UnderwaterPass.js`
  L27–L94 · L112–L184 · L187–L225
- `src/scene/floaters.js`
  L4–L38 · L111–L135 · L111–L178 · L181–L201
- `README.md`
  L51–L59 · L61–L97 · L99–L117 · L151–L156
- `src/water/WaterQuery.js`
  L5–L58
- `src/engine/gpuTimer.js`
  L1–L69
- `src/water/shoreMap.js`
  L75–L86 · L258–L271

状态：12 问均有固定源码依据；本地自建 14 项数值检查 PASS，GPU=NOT_TESTED。浏览器视觉对照及一次内置基准已记录，CPU检查并不证明GPU数值正确；链接固定 SHA，不随分支更新漂移。